



Semaine 1 : Variables, types et console

Objectif : Apprendre à afficher des messages et stocker des informations

Durée : 2 heures

Lien Scratch : Les variables oranges 🟡



Ce que tu vas apprendre

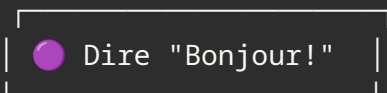
1. Afficher des messages avec `console.log()`
2. Créer des variables pour stocker des valeurs
3. Les différents types de données
4. Faire des calculs

1 Afficher des messages : `console.log()`

En Scratch, tu utilises le bloc **"Dire"** pour afficher quelque chose.

En JavaScript, on utilise `console.log()`.

Scratch vs JavaScript

 ↔ `console.log("Bonjour!");`

Exemples

```
// Afficher du texte
console.log("Salut, je suis un programme !");

// Afficher un nombre
console.log(42);

// Afficher un calcul
console.log(2 + 2);

// Afficher plusieurs choses
console.log("Le résultat est :", 5 + 3);
```



À retenir

- Le texte doit être entre guillemets `"..."` ou `'...'`
- Les nombres s'écrivent sans guillemets
- Le `;` à la fin est optionnel mais recommandé
- `//` permet d'écrire des commentaires (le programme les ignore)



Exercice : `exercice_1_console_log.js`

2 Les variables : stocker des informations

En Scratch, tu crées des variables avec le menu **Variables** (blocs oranges).

En JavaScript, on utilise `let` .

Scratch vs JavaScript

● Mettre "score" à 0

↔ `let score = 0;`

● Ajouter 10 à "score"

↔ `score = score + 10;`

Créer une variable

```
// Créer une variable avec une valeur
let score = 0;
let nomDuJoueur = "Alex";
let vies = 3;

// Afficher la variable
console.log(score);           // Affiche: 0
console.log(nomDuJoueur);     // Affiche: Alex
```

Modifier une variable

```
let score = 0;
console.log(score);           // Affiche: 0

score = 100;                  // On change la valeur
console.log(score);           // Affiche: 100

score = score + 50;           // On ajoute 50
console.log(score);           // Affiche: 150
```

Raccourcis pratiques

```
let score = 10;

score += 5; // Pareil que: score = score + 5 → score vaut 15
score -= 3; // Pareil que: score = score - 3 → score vaut 12
score *= 2; // Pareil que: score = score * 2 → score vaut 24
score /= 4; // Pareil que: score = score / 4 → score vaut 6
```

let vs const

```
let score = 0; // Peut changer (let = "laisser")
const PI = 3.14159; // Ne peut PAS changer (const = "constante")

score = 100; // ✅ OK
PI = 3; // ❌ ERREUR ! Une constante ne peut pas changer
```

Règle simple : Utilise `const` si la valeur ne changera jamais, sinon `let` .

 **Exercice** : `exercice_2_variables.js`

3 Les types de données

En JavaScript, il y a plusieurs types de valeurs :

Type	Exemple	Description
string (texte)	"Bonjour" , 'Hello'	Texte entre guillemets
number (nombre)	42 , 3.14 , -5	Nombres entiers ou décimaux
boolean (booléen)	true , false	Vrai ou Faux

Exemples

```
// String (texte)
let prenom = "Lucas";
let message = 'Bienvenue dans le jeu !';

// Number (nombre)
let age = 12;
let prix = 19.99;
let temperature = -5;

// Boolean (vrai/faux)
let estConnecte = true;
let aPerdu = false;
```

Connaître le type d'une variable

```
let nom = "Alice";
let age = 12;
let actif = true;

console.log(typeof nom);    // Affiche: string
console.log(typeof age);    // Affiche: number
console.log(typeof actif);  // Affiche: boolean
```



Exercice : `exercice_3_types.js`

4 Les opérations mathématiques

Opérations de base

```
let a = 10;
let b = 3;

console.log(a + b); // Addition      → 13
console.log(a - b); // Soustraction  → 7
console.log(a * b); // Multiplication → 30
console.log(a / b); // Division      → 3.333...
console.log(a % b); // Modulo (reste) → 1
console.log(a ** b); // Puissance    → 1000 (103)
```

Le modulo % (très utile !)

Le modulo donne le **reste** de la division :

```
console.log(10 % 3); // 10 ÷ 3 = 3 reste 1 → Affiche: 1
console.log(15 % 5); // 15 ÷ 5 = 3 reste 0 → Affiche: 0
console.log(7 % 2);  // 7 ÷ 2 = 3 reste 1  → Affiche: 1
```

Astuce : `nombre % 2` permet de savoir si un nombre est pair (reste 0) ou impair (reste 1).



Exercices : `exercice_4_calculs.js` et `exercice_7_modulo.js`

Concaténer du texte

On peut "coller" du texte avec `+` :

```
let prenom = "Lucas";
let age = 12;

console.log("Bonjour " + prenom); // Bonjour Lucas
console.log(prenom + " a " + age + " ans"); // Lucas a 12 ans
```

Template literals (plus pratique !)

Avec les backticks ``` et `${}`, c'est plus facile :

```
let prenom = "Lucas";
let age = 12;

console.log(`Bonjour ${prenom}`);           // Bonjour Lucas
console.log(`${prenom} a ${age} ans`);      // Lucas a 12 ans
console.log(`Dans 10 ans, tu auras ${age + 10} ans`);
```

 **Exercices :** `exercice_5_modifier_variables.js` et `exercice_6_texte.js`

5 Demander une entrée utilisateur

Pour demander quelque chose à l'utilisateur en Node.js, on utilise un module spécial.

Pour l'instant, on va utiliser une version simplifiée :

Méthode simple avec `process.argv`

Quand tu lances le programme, tu peux passer des informations :

```
// fichier: saluer.js
// Le 3ème argument (après node et le fichier)
let prenom = process.argv[2];

console.log(`Bonjour ${prenom} !`);
```

Exécution :

```
node saluer.js Lucas
# Affiche: Bonjour Lucas !
```

Méthode interactive (pour plus tard)

On verra comment faire une vraie interaction question/réponse dans les prochaines semaines !



Exercice : `exercice_8_entree_utilisateur.js`



Résumé

Concept	Syntaxe
Afficher	<code>console.log("texte");</code>
Variable modifiable	<code>let nom = valeur;</code>
Constante	<code>const NOM = valeur;</code>
Texte	<code>"entre guillemets"</code> ou <code>`avec backticks`</code>
Nombre	<code>42</code> , <code>3.14</code>
Booléen	<code>true</code> , <code>false</code>
Calculs	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>%</code> , <code>**</code>
Template string	<code>`Bonjour \${variable}`</code>



Mini-projet : Calculatrice parlante

À la fin de cette séance, tu sauras créer une calculatrice qui affiche :



CALCULATRICE PARLANTE



Nombre 1 : 15

Nombre 2 : 7

Addition : $15 + 7 = 22$

Soustraction : $15 - 7 = 8$

Multiplication : $15 \times 7 = 105$

Division : $15 \div 7 = 2.14$



Prochaine étape

1. Termine par le défi bonus : `defi_fiche_joueur.js`
2. Semaine prochaine : les conditions (if/else) !